

Backend Developer — Easy Guide

Crash course before the presentation · Retail Demand Forecasting (Walmart M5)

In one sentence

I built the **FastAPI** server that sits between the browser and our data — 34 endpoints, 149 tests, and it never lets the Gemini key reach the browser.

The words, first

API — Application Programming Interface. A defined way for two programs to talk. Not a user interface — no buttons. Our React app asks our FastAPI server for forecast data.

HTTP. The language of the request. It has a **method** (GET = fetch, POST = send something), a **path** (/api/v1/health), and comes back with a **status code** (200 fine, 404 not found, 503 unavailable) plus a body.

JSON. The format of that body — {"key": value} text that both Python and JavaScript read natively.

Endpoint / route. One address the API answers on, e.g. /api/v1/series/CA_3/F00DS_3_090/forecast.

FastAPI. The Python framework. It validates requests automatically and generates live docs at /docs from the code itself.

Uvicorn. The server process that runs FastAPI and listens on a port.

The picture

```
BROWSER (React)
| GET /api/v1/series/CA_3/F00DS_3_090/forecast
v
FastAPI
|-- routers/      handle HTTP: validate, pick the status code
|-- services/     the logic and the SQL
|-- db.py         DuckDB, opened READ-ONLY
v
product.duckdb + history.parquet + backtest.parquet
|
v
BROWSER <-- 200 + JSON --> chart is drawn
```

Three layers on purpose: a router never contains SQL, a service never knows about status codes. When something breaks, you know which file to open.

The 34 endpoints, grouped

Group	What it answers
Health (2)	/health (alive?), /ready (can it serve?)
Meta (3)	model card, capability matrix, provenance
Hierarchy (4)	levels, nodes, search, roll-up aggregates
Series (4)	list, detail, history, forecast
Accuracy (10)	windows, levels, horizon, regimes, occurrence, members...
Insights (3)	summary, top movers, planning view
Inference (4)	status, jobs, verify the frozen model
GenAI (4)	status, suggestions, ask, context-preview

All under /api/v1 — versioned, so a future /api/v2 couldn't break anything.

Two endpoints you should be able to walk through

GET /api/v1/series/{store}/{item}/forecast

- **Frontend sends:** store and item in the URL
- **Backend does:** validates them, looks up the frozen forecast in DuckDB, attaches the error band for that product's demand regime
- **Returns:** 28 daily values, the 28-day total, a planning range, and a band_basis field saying in words that the range is **observed past error**

Verified example: CA_3 / FOODS_3_090 → total_28d = 3331.3681

POST /api/v1/genai/ask

- **Frontend sends:** {"question": "...", "store_id": "...", "item_id": "..."}
- **Backend does:** checks local policy first (an attack is refused with ****no AI call****), otherwise gathers verified numbers and asks Gemini, then checks every number in the reply
- **Returns:** the answer plus grounded, refused, intent, elapsed_ms

Health vs Ready — know the difference

	/health	/ready
Means	the process is running	it can actually serve
Touches data?	no	yes — opens DuckDB, counts tables

Our container healthcheck uses /ready, deliberately: a process that's up but has no data volume is alive and useless.

The big design decision

The backend does NOT run the model on every request.

The forecast was computed once during research and stored. A request is a database lookup — **8–13 ms**. That's why the API starts in under a second and needs no ML libraries.

There's a **separate optional** endpoint, POST /api/v1/inference/verify, that really does reload the frozen models and re-run everything. It takes ~48 s, so it runs as a background job and the frontend polls for the result.

VERIFIED: max_abs_diff = 0.000e+00 — *the live model reproduces the shipped forecast exactly.*

Security, in four layers

- **DuckDB opened read-only** — the API cannot write to the data
- **Model and data mounted** :ro in the container
- **Non-root container user** (uid 10001)
- **Whitelisting, not sanitising** — level names must be in a fixed SAFE_LEVELS

set or the request is rejected before any SQL is built

SQL injection = sending SQL as data so it gets executed. A whitelist makes it structurally impossible. There's a test that fires injection strings at it.

The Gemini key is an environment variable, held server-side as a masked secret. Never in code, never in a prompt, never in a response, never in the browser bundle.

Configuration — nothing hard-coded

Every setting is an environment variable with an NPN_ prefix: NPN_DATA_DIR, NPN_PROJECT_ROOT, NPN_ENABLE_INFERENCE, NPN_CORS_ORIGINS, plus GEMINI_API_KEY.

A test asserts there is **no hard-coded Windows path** in the app, and another proves the API works with the **research tree entirely absent**.

CORS — Cross-Origin Resource Sharing. A browser rule about calling a different origin. We configure it, but production doesn't use it: nginx serves the app and proxies /api/, so the browser sees one origin. It only matters in development.

Tests: 149 passing

File	Tests
test_genai.py	71
test_hierarchy.py	16
test_accuracy.py	12
test_integrity.py	10 — the freeze guard
test_series.py	10
others	30

The interesting ones aren't HTTP checks — they assert that **children sum to their parent**, that the **served forecast still matches the frozen file**, and that **no accuracy is ever returned for a window with no ground truth**.

How to explain it to a judge

"I built the backend. It's FastAPI — a Python framework for building APIs. An API is just a defined way for two programs to talk: the React app sends an HTTP request like 'give me the forecast for this product in this store' and gets JSON back.

There are 34 endpoints in three layers — routers handle HTTP, services hold the logic and SQL, and a database module opens DuckDB read-only.

The key decision is that we don't run the model on every request. The forecast was computed once and stored, so a request is just a lookup — 8 to 13 milliseconds. There's a separate endpoint that actually reloads the frozen model and re-runs all 853,720 predictions to prove it still matches exactly.

For security: read-only database, read-only mounts, non-root container, and level names are whitelisted so SQL injection isn't possible. The Gemini key stays on the server — the browser never sees it.

149 tests, and the ones I'd point at check that store totals equal the sum of their products, and that what we serve still matches the frozen forecast file."

Quick answers

What is an API? A defined way for two programs to talk — ours uses HTTP + JSON. *Why FastAPI?* Automatic validation and auto-generated docs that can't drift. *Does it run the model per request?* No — it reads a stored forecast. Model re-runs only on the verify endpoint. *How do you stop SQL injection?* Whitelist of allowed level names; anything else is rejected before SQL is built. *Why /ready for healthchecks?* Health = alive. Ready = can serve. *Does it need the research data?* No — proven by a test with an empty project root. *Where's the Gemini key?* Server-side environment variable only. Never in the browser.

30-second revision

FastAPI · **34 endpoints** under `/api/v1` · routers → services → db.py · **DuckDB read-only** · forecast is a **lookup, 8–13 ms**, not a model run · `/health` alive vs `/ready` can-serve · whitelisting stops SQL injection · non-root + `:ro` mounts · key is server-side only · **149 tests** · verify endpoint proves `max_abs_diff = 0.000e+00`.